

Predicción de cancelación de reservas de hotel

Práctica final de Machine Learning: un sistema que entrena y compara cinco modelos de clasificación (más un modelo base de referencia) para predecir si una reserva de hotel se va a cancelar, elige el mejor y lo deja guardado para hacer predicciones nuevas.

Máster en IA, Cloud Computing y DevOps · Machine Learning y Deep Learning · Práctica de evaluación final

Repositorio: <https://github.com/tmllabres/MasterPonita-EntregaFinal-Grupo6-ML>

Este README es también el informe final: docs/informe_final.pdf es este mismo fichero exportado a PDF.

1. Autor y roles

Autor	Correo	Usuario de GitHub
Antonio Martínez Llabrés	tmllabres@gmail.com	tmllabres

La práctica se planteó por parejas, pero el otro integrante la dejó antes de la entrega, y así se lo comuniqué al profesor el 29 de septiembre de 2026. Es un trabajo individual: no hay partes de otro compañero, y el historial de commits lo refleja.

Parte	Ficheros
Datos: carga, limpieza, partición y preprocesado	src/data_loader.py, tests/test_data_loader.py
Análisis exploratorio	notebooks/exploracion/eda_inicial.ipynb, notebooks/finales/eda_final.ipynb
Modelos: registro, comparación y búsqueda	src/model_trainer.py, tests/test_model_trainer.py, notebooks/exploracion/pruebas_modelos.ipynb
Evaluación y figuras	src/evaluator.py, tests/test_evaluator.py, notebooks/finales/comparativa_modelos.ipynb
Inferencia	src/predictor.py, tests/test_predictor.py
Orquestación y configuración	main.py, src/config.py, tests/test_contrato.py
Documentación	README.md, docs/

2. El problema y los datos

Qué quiero predecir: si una reserva se va a cancelar (`is_canceled = 1`) o no (`0`). Es un problema de clasificación binaria supervisada.

Los datos: el CSV de la práctica, con 119.390 reservas de dos hoteles (uno urbano y otro tipo resort) y 32 columnas. En el CSV original cancela el 37,04 % de las reservas. Las variables están explicadas en [docs/diccionario_datos.md](#). Me parece un buen dataset para este problema porque son reservas reales, trae la respuesta (`is_canceled`) y casi todo lo que el hotel ya sabe al hacerse la reserva: la antelación, el canal, el agente, el país o las peticiones especiales.

Para qué sirve

Lo he planteado pensando en el departamento del hotel que gestiona las reservas y los precios. Cuando una reserva se cancela a última hora, la habitación se queda vacía y esa noche ya no se puede vender. Si el hotel sabe de antemano qué reservas tienen más riesgo de cancelarse, puede hacer overbooking con cuidado, pedir un depósito a esas reservas o prepararse para los días con más riesgo.

Sin un modelo, el hotel no puede distinguir unas reservas de otras y las trata todas igual. Para comparar, uso un modelo base (DummyClassifier) que hace justo eso: dice siempre «no cancela», que es lo más frecuente, sin mirar ningún dato de la reserva.

Los dos errores posibles

Error	Qué pasa	Qué le cuesta al hotel
Falso negativo: digo que no cancela y sí cancela	La habitación se queda vacía sin que nadie lo espere	La noche entera, que ya no se recupera
Falso positivo: digo que cancela y el cliente viene	El hotel ha revendido una habitación que sí se iba a usar	Hay que realojar al cliente y compensarle, y queda mal ante él

Los dos errores cuestan dinero, pero el dataset no dice cuánto cuesta cada uno. Por eso elegí una métrica que tiene en cuenta los dos a la vez (apartado 5).

Fugas de datos

Una fuga es una columna que da la respuesta «por la puerta de atrás»: una información que el modelo no tendría en el momento de predecir. Yo quiero predecir al hacerse la reserva o en las semanas siguientes, antes de que llegue el cliente.

- **reservation_status y reservation_status_date:** la primera dice directamente si la reserva se canceló (Canceled o No-Show = 1, Check-Out = 0), y la segunda es la fecha de ese estado (el día en que se canceló o el día en que el cliente se fue), así que también da la respuesta. Con ellas cualquier modelo acierta casi el 100 %, así que las quité desde el principio.
- **required_car_parking_spaces y assigned_room_type:** estas me costó más verlas. De las 7.416 reservas con plaza de parking, ninguna está cancelada, lo que tiene sentido si la plaza se anota cuando el cliente llega con el coche. Con la habitación asignada pasa algo parecido: cambia respecto a la reservada en el 18,8 % de las reservas que se completan y en el 17,2 % de los No-Show, pero solo en el 1,4 % de las que se cancelaron antes de llegar. Como los No-Show también la tienen, deduzco que la habitación se asigna el día de la llegada, y quien cancela antes nunca llega a ese día.

Para medir cuánto inflaban el resultado usé un modelo rápido de prueba (el HistGradientBoostingClassifier de scikit-learn con sus valores por defecto, en pruebas_modelos.ipynb): con estas dos columnas su F1 subía de 0,678 a 0,710, pero era una mejora falsa, así que también las quité.

Me quedan 27 columnas para predecir (32 – la respuesta – 4 fugas). booking_changes (número de cambios en la reserva) me generó dudas, porque el CSV guarda el número final de cambios, y algunos podrían ser posteriores al momento de predecir. La dejé porque no se comporta como el parking: también tienen cambios muchas reservas que nunca llegaron al hotel (el 14,7 % de los No-Show), así que no se rellena solo al llegar. Además, con el modelo de prueba, sin ella el F1 solo baja de 0,678 a 0,671, así que aunque una parte fuera fuga pesaría poco.

Limpieza

- 1. **Duplicados:** después de quitar las 4 columnas de fuga, hay 33.413 filas repetidas exactamente (un 28 % del CSV). Si se miran también las columnas de fuga salen 31.994, que es la cifra de `eda_inicial.ipynb`; uso la primera porque el modelo no ve esas columnas. El CSV no tiene un identificador de reserva, así que no se pueden distinguir entre sí. Las quité antes de separar train y test, porque si una copia cae en train y otra en test, el modelo se examina de algo que ya ha visto. Lo comprobé: sin quitarlas, el 32,7 % de las filas de test tendrían una copia exacta en train.
- 2. **Registros imposibles:** quité 1 reserva con precio negativo y 165 reservas sin ningún huésped (0 adultos, 0 niños y 0 bebés). En el CSV original hay 180 sin huéspedes, que es la cifra de `eda_inicial.ipynb`, pero 15 ya se habían ido con los duplicados.
- 3. **Lo que no quité:** las reservas con precio 0 (hay invitaciones del hotel) y una reserva con un precio de 5.400 por noche, que es rarísima pero no imposible.

Después de limpiar quedan **85.811 reservas**, de las que cancela el **27,64 %**. Las separo en **68.648 de entrenamiento y 17.163 de test** (80/20), manteniendo el mismo porcentaje de cancelaciones en las dos partes (stratify).

El porcentaje de cancelaciones baja del 37,04 % al 27,64 % porque el 61 % de las filas repetidas eran cancelaciones. Por eso el modelo base acierta ahora el 72,36 % (apartado 5).

Quitar los duplicados tuvo un efecto que asumo: muchas de las copias eran reservas de grupos, y casi desaparecen las reservas con tarifa no reembolsable (Non Refund).

3. Análisis exploratorio (EDA)

El análisis completo está en [notebooks/finales/eda_final.ipynb](#). Esta tabla resume lo que encontré y qué decisión tomé en cada caso:

Hallazgo	Evidencia	Decisión
Clases desbalanceadas	Tras limpiar, 72,36 % no cancela (62,96 % en crudo)	F1 de la clase «cancela» como métrica principal; accuracy, secundaria
reservation_status es la respuesta	Check-Out → 0; Canceled y No-Show → 1, sin excepciones	Se elimina, junto con su fecha
Parking y habitación asignada se rellenan al llegar	7.416 reservas con plaza: ninguna cancelada ni No-Show	Se eliminan: quedan 27 predictoras
Duplicados exactos (sin contar las fugas)	33.413 filas (28,0 %); sin quitarlos, el 32,7 % del test tendría una gemela en train	Se eliminan antes de partir: 85.811 filas
Registros imposibles	1 reserva con adr negativo y 165 sin ningún huésped	Se eliminan
Alta cardinalidad	country, agent y company: 168, 325, 320 valores; one-hot ingenuo de 880 columnas	Top-10 + cajón dentro del Pipeline: 97 columnas
Nulos con significado	agent 13,7 % y company 94,1 % nulos en train	El nulo es su propia categoría: sin agente = reserva directa
lead_time separa	Mediana de 79 días si cancela frente a 37 si no	Se mantiene; mediana para imputar y escalado, dentro del Pipeline

4. Diseño del sistema

He separado el código en módulos, cada uno con una tarea, y `main.py` los llama en orden:

```
MasterPonita-EntregaFinal-Grupo6-ML/
├─ main.py                ejecuta todo el proceso: python main.py
├─ requirements.txt        librerías con la versión fijada
├─ .python-version         3.12
├─ .gitignore
├─
├─ src/
│   ├── config.py          parámetros: rutas, semilla, métrica, umbral...
│   ├── data_loader.py     carga, limpieza, partición y preprocesado
│   ├── model_trainer.py   los modelos y su comparación
│   ├── evaluator.py       métricas y figuras sobre el test
│   └─ predictor.py        predicciones con el modelo guardado
├─
├─ notebooks/
│   ├── exploracion/       pruebas y borradores
│   │   ├── eda_inicial.ipynb
│   │   └─ pruebas_modelos.ipynb
│   └─ finales/            los notebooks que presento
│       ├── eda_final.ipynb
│       └─ comparativa_modelos.ipynb
├─
├─ tests/                  tests con pytest
├─ data/raw/               el CSV original
├─ models/                 el modelo entrenado (no se sube a GitHub)
├─ outputs/                figuras y tablas de resultados
└─ docs/                   diccionario de datos, enunciado e informe en PDF
```

Pasos de `main.py`:

1. Carga el CSV, lo limpia, lo separa en train y test y prepara el preprocesado (`data_loader.preparar()`).
2. Compara los seis modelos con validación cruzada de 5 particiones (folds) sobre el train, buscando antes los mejores hiperparámetros de los cuatro que los tienen. Luego vuelve a entrenar cada uno con todo el train y hace una tabla comparativa (`model_trainer.entrenar_y_comparar()`).
3. Elige el mejor modelo según el F1 (`model_trainer.elegir_mejor()`).
4. Evalúa ese modelo una sola vez sobre el test y genera las figuras (`evaluator`).
5. Guarda el modelo en `models/` (`model_trainer.guardar()`).
6. Lo vuelve a cargar desde disco y predice unas cuantas reservas, para comprobar que el modelo guardado funciona (`predictor`).

Decisiones que tomé:

- **Uso un Pipeline de scikit-learn con el preprocesado dentro.** Así, en cada fold de la validación cruzada el preprocesado (medianas, escalado, categorías) se aprende solo con los datos de entrenamiento de ese fold, y no se cuela información de la parte de validación.
- **El preprocesado** trata distinto cada tipo de columna: a las numéricas les relleno los huecos con la mediana y las escalo; a las categóricas les aplico one-hot. Tres columnas (`country`, `agent` y `company`) tienen cientos de valores distintos, así que me quedo con sus 10 valores más frecuentes y agrupo el resto en «otros». Con eso paso de 880 columnas a 97.
- **Todos los modelos siguen la misma estructura** (una clase base, `ModeloBase`, con `construir()` y `espacio_busqueda()`), y un mismo bucle los entrena y los compara a todos, así que se comparan en igualdad de

condiciones. La idea la saqué de las librerías de AutoML que menciona el enunciado (PyCaret, H2O...): añadir otro modelo es escribir una clase y añadirla al registro.

- **La red neuronal se crea de nuevo en cada entrenamiento.** Si se creara una sola vez, la validación cruzada reutilizaría la misma red entre folds sin reiniciar los pesos, y el resultado saldría inflado. Tengo un test que comprueba que dos entrenamientos seguidos dan exactamente lo mismo.
- **Los parámetros generales están en src/config.py** (rutas, semilla, métrica, umbral, número de folds y modo demo), y uso siempre la misma semilla (42), así que al repetir la ejecución salen los mismos resultados. Las combinaciones de hiperparámetros que prueba la búsqueda están en cada modelo, en `src/model_trainer.py` (apartado 11).
- **Guardo el Pipeline completo**, no solo el modelo, junto con el umbral y las columnas con los que se entrenó. Así, para predecir basta con pasarle una reserva con las columnas originales.

5. Métrica principal y por qué

Métrica principal: F1 de la clase «cancela». **Secundarias:** accuracy, precision, recall y ROC-AUC. La elegí antes de entrenar y está fijada en `src/config.py`, para no escogerla después según qué modelo saliera mejor.

Por qué no accuracy. Un modelo que dijera siempre «no cancela» acertaría el 72,36 % sin haber aprendido nada, porque la mayoría de reservas no se cancela. Es lo que saca el modelo base en la tabla del apartado 7: accuracy 0,724 pero F1 0. Con accuracy, casi cualquier modelo parece bueno.

Por qué F1. Como expliqué en el apartado 2, los dos errores le cuestan dinero al hotel: no detectar una cancelación deja una habitación vacía, y dar por cancelada una reserva que no lo es obliga a realojar al cliente. Si solo mirara el recall, el modelo tendería a marcar demasiadas reservas como canceladas; si solo mirara la precision, solo marcaría las más obvias. El F1 combina las dos, y como no sé cuánto cuesta cada error, me pareció lo más razonable.

ROC-AUC lo uso como apoyo: mide lo bien que el modelo ordena las reservas de más a menos riesgo, pero el hotel necesita una respuesta sí/no para cada reserva, y eso es lo que mide el F1 (con un umbral de 0,5).

6. Cómo ejecutar el proyecto

Requisitos: Python 3.12. Todas las librerías, con su versión fijada, están en `requirements.txt`.

```
git clone https://github.com/tmllabres/MasterPonita-EntregaFinal-Grupo6-ML.git
cd MasterPonita-EntregaFinal-Grupo6-ML

python -m venv .venv          # con Python 3.12 (compruébalo con python --version)
# Windows con varias versiones instaladas: py -3.12 -m venv .venv
.venv\Scripts\activate        # Windows
# source .venv/bin/activate    # macOS / Linux

python -m pip install -r requirements.txt
```

En Windows, clona en una ruta corta (por ejemplo `C:\proyectos\`): TensorFlow instala ficheros con rutas internas muy largas y, en una carpeta muy anidada, pip falla con «El nombre del archivo o la extensión es demasiado largo».

▼ Alternativa con uv (opcional)

Con [uv](#) la instalación es bastante más rápida:

```
uv venv --python 3.12
uv pip install -r requirements.txt
```

Tests (unos 120, tardan en torno a un minuto):

```
python -m pytest -q
```

Proceso completo (tarda unos 10 minutos, sobre todo por la búsqueda de hiperparámetros del Random Forest):

```
python main.py
```

Versión corta para la defensa (usa 20.000 reservas, 3 folds y menos épocas en la red; tarda en torno a un minuto y medio):

```
python main.py --demo
```

La demo guarda sus resultados en las mismas carpetas que el proceso completo, así que sobrescribe `outputs/` y el modelo de `models/`. Las figuras y tablas oficiales se recuperan con `git restore outputs/`; el modelo oficial, volviendo a ejecutar `python main.py`.

Predecir con el modelo ya entrenado:

```
python -m src.predictor
```

La carpeta `models/` no se sube a GitHub, así que en un clon nuevo hay que ejecutar `python main.py` (o `--demo`) al menos una vez antes de predecir.

Notebooks: `jupyter lab` desde la carpeta del proyecto. `comparativa_modelos.ipynb` lee los resultados que ya están en `outputs/`, así que no hace falta ejecutar antes `main.py`.

Informe en PDF: `python docs/exportar_informe.py` vuelve a generar `docs/informe_final.pdf` a partir de este README (necesita Chrome o Edge).

7. Modelos comparados

He comparado los cinco modelos que pide el enunciado y un modelo base (`DummyClassifier`, que siempre dice «no cancela»). Todos se evalúan igual: validación cruzada de 5 folds con la misma semilla sobre las 68.648 reservas de entrenamiento. Para cada modelo, menos la red (que es muy lenta) y el modelo base (que no tiene nada que ajustar), busqué los mejores hiperparámetros con `GridSearchCV` (apartado 11).

La tabla sale de outputs/tabla_comparativa.csv y muestra la media $\pm$ la desviación de los 5 folds. El tiempo incluye la búsqueda de hiperparámetros.

Modelo	F1 (CV)	Accuracy	Precision	Recall	ROC-AUC	Tiempo (s)
XGBoost	0,697 $\pm$ 0,007	0,842 $\pm$ 0,003	0,740 $\pm$ 0,006	0,659 $\pm$ 0,008	0,903 $\pm$ 0,004	100
Random Forest	0,693 $\pm$ 0,006	0,841 $\pm$ 0,003	0,742 $\pm$ 0,004	0,650 $\pm$ 0,007	0,898 $\pm$ 0,004	374
Red neuronal (Keras)	0,669 $\pm$ 0,012	0,832 $\pm$ 0,004	0,736 $\pm$ 0,005	0,614 $\pm$ 0,019	0,891 $\pm$ 0,004	100
Árbol de decisión	0,651 $\pm$ 0,003	0,819 $\pm$ 0,002	0,696 $\pm$ 0,008	0,611 $\pm$ 0,006	0,865 $\pm$ 0,006	13
Regresión logística	0,542 $\pm$ 0,006	0,786 $\pm$ 0,003	0,663 $\pm$ 0,008	0,458 $\pm$ 0,007	0,831 $\pm$ 0,004	11
Baseline (Dummy)	0,000 $\pm$ 0,000	0,724 $\pm$ 0,000	0,000 $\pm$ 0,000	0,000 $\pm$ 0,000	0,500 $\pm$ 0,000	2

Con el modelo base se ve bien el problema de la accuracy: todos los modelos quedan entre el 72 % y el 84 %, mientras que en F1 las diferencias son mucho más claras.

8. Resultados y elección final

Modelo elegido: XGBoost. La búsqueda eligió 800 árboles, learning_rate de 0,05 y profundidad máxima de 8. Además, cada árbol usa un 80 % de las filas y de las columnas; ese valor lo fijé yo y no entró en la búsqueda.

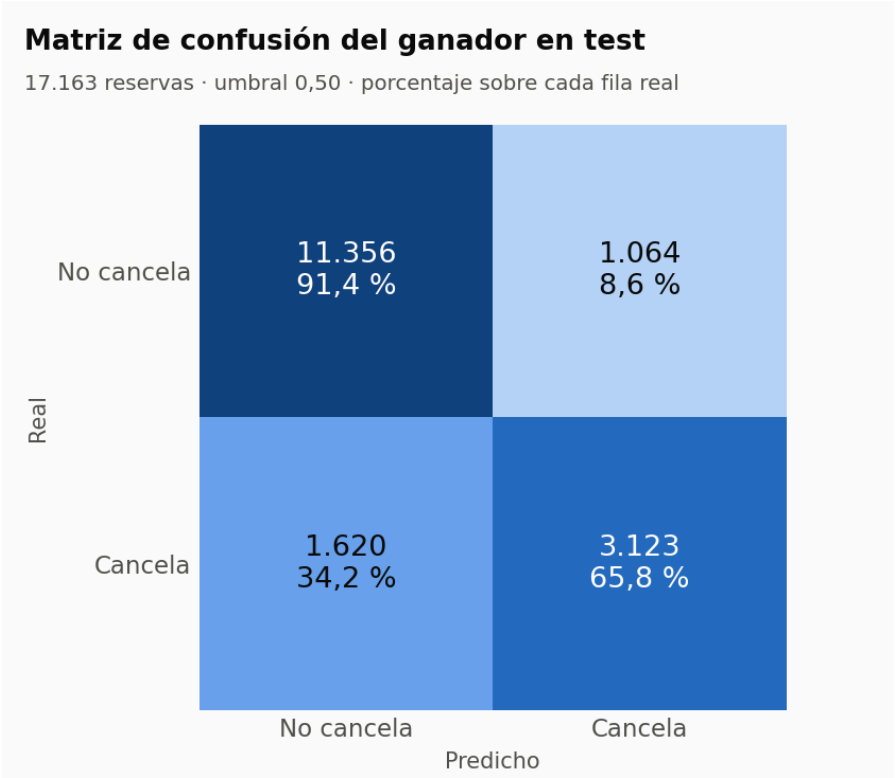
Por qué gana:

- Tiene el mejor F1 en la validación cruzada (0,697) y en el test da un resultado parecido (0,699), así que no parece que haya tenido suerte con los folds.
- El Random Forest queda muy cerca (0,693). XGBoost le gana en F1, recall y AUC, y además tardó bastante menos. Los dos combinan muchos árboles, pero en XGBoost cada árbol intenta corregir los errores de los anteriores.
- La regresión logística queda muy por debajo (0,542). Creo que es porque no puede combinar variables: por ejemplo, que una reserva hecha con mucha antelación pese distinto según el agente o el tipo de cliente. Un solo árbol de decisión (0,651) tampoco llega al nivel de los modelos que combinan muchos árboles.
- La red neuronal (0,669) queda en medio: con datos en forma de tabla y con muchas categorías, los modelos de árboles suelen funcionar mejor. Además es la que más varía entre folds, y cada entrenamiento suyo es el más lento de todos. En la tabla tarda lo mismo que XGBoost, pero porque la red entrena una sola combinación y XGBoost prueba 27.

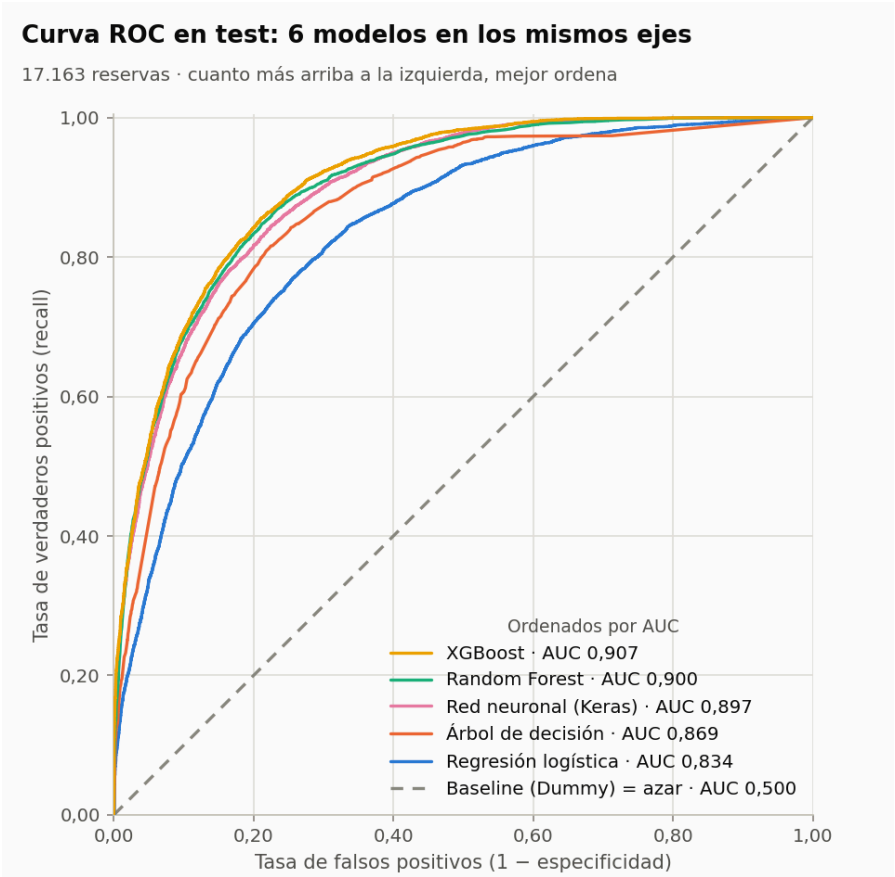
Resultados del ganador en el test (17.163 reservas que no se usaron para entrenar; los valores están en outputs/metricas_test.json):

Métrica	Valor
F1	0,699
Accuracy	0,844
Precision	0,746
Recall	0,658
ROC-AUC	0,907

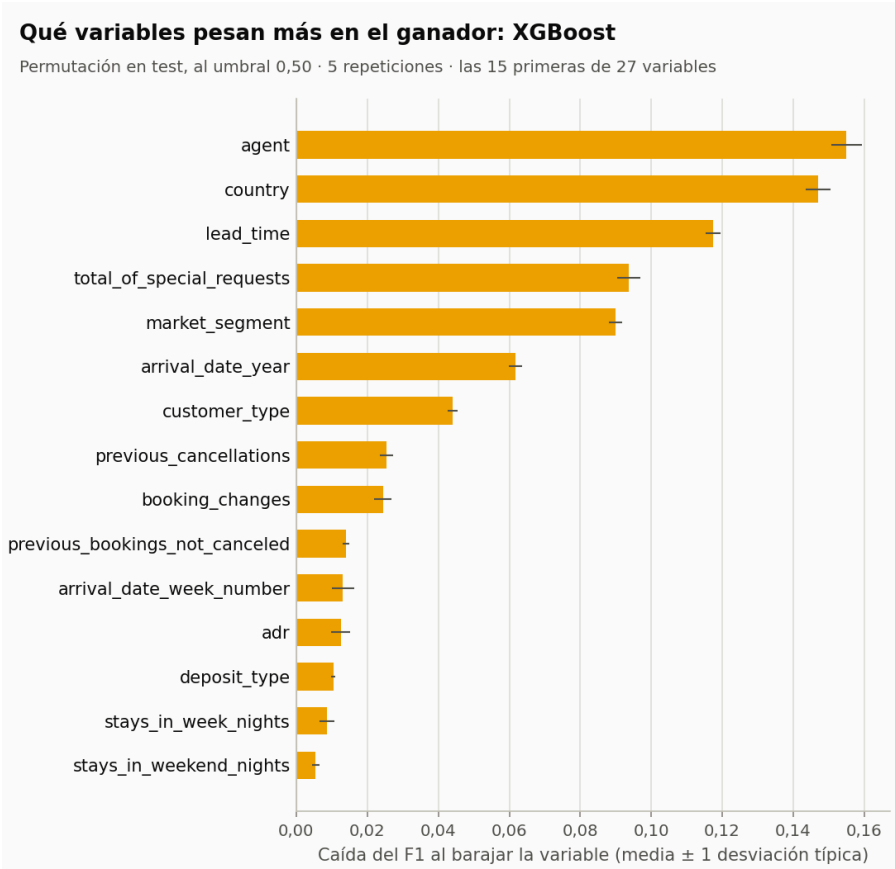
Matriz de confusión. De las 4.743 reservas del test que se cancelaron, el modelo detecta 3.123 (el 65,8 %) y se le escapan 1.620. De las 12.420 que no se cancelaron, marca por error 1.064 (el 8,6 %).



Curva ROC. Pongo los seis modelos en el mismo gráfico. XGBoost, el Random Forest y la red quedan muy juntos (AUC entre 0,897 y 0,907), el árbol (0,869) y la regresión logística (0,834) quedan por debajo y el modelo base es la diagonal.



Importancia de las variables. Para medirla uso la importancia por permutación sobre el test: se desordena una variable y se mira cuánto empeora el F1. Así funciona igual con cualquier modelo y sale por variable original (las 27), no por cada una de las 97 columnas del one-hot. Las que más pesan son agent, country, lead_time, total_of_special_requests y market_segment. Algunas variables, como hotel, salen casi a cero, probablemente porque su información ya está en otras (por ejemplo, en el agente).



9. Conclusiones

- El modelo es útil para el hotel: de cada cuatro reservas que marca como canceladas, tres se cancelan de verdad (precision 0,746), y detecta dos de cada tres cancelaciones (recall 0,658). Con esto se podría pedir un depósito o hacer overbooking solo con las reservas de más riesgo, en lugar de tratar a todas igual.
- Además de la predicción sí/no, el modelo da una probabilidad de cancelación para cada reserva, y con ella ordena bien las reservas de más a menos riesgo (AUC 0,907). Así el hotel puede empezar por las más arriesgadas.
- Lo que más me enseñó la práctica es la importancia de preparar bien los datos. Entre los tres mejores modelos la diferencia de F1 es pequeña (de 0,669 a 0,697), menos de lo que subía el F1 del modelo de prueba al dejar las dos fugas de la llegada (de 0,678 a 0,710). Y sin quitar los duplicados, un tercio del test (32,7 %) tendría una copia exacta en train. Los resultados habrían salido mejores, pero no serían reales.
- Las variables que más ayudan a predecir son el agente de la reserva, el país, la antelación con la que se reserva y las peticiones especiales.

10. Reflexión crítica: limitaciones y mejoras

- **La validación cruzada no es anidada.** Los hiperparámetros se eligen con los mismos folds con los que luego se mide, así que el F1 de la tabla puede salir algo optimista. Por eso la cifra final es la del test, que solo se usa una vez. Se podría mejorar con una validación cruzada anidada.
- **La partición es aleatoria y no por fechas.** En los datos ya limpios, las cancelaciones aumentan cada año (20,6 % en 2015, 26,5 % en 2016 y 32,0 % en 2017), y en la realidad el modelo tendría que predecir reservas futuras. Además, `arrival_date_year` es la sexta variable más importante del modelo. Una prueba más realista sería entrenar con 2015-2016 y evaluar con 2017, y probablemente saldría algo peor.
- **El umbral de 0,5 no tiene en cuenta el coste real** de cada error, porque el dataset no lo incluye. Con esos costes se podría elegir un umbral mejor.
- **country podría ser en parte una fuga.** Entre los No-Show hay muchas más reservas de Portugal (59,4 %) que entre las que sí llegaron (27,6 %), y eso me hace sospechar que el país se corrige al hacer el check-in. No la quité porque la prueba no es tan clara como la del parking: allí ninguna reserva con plaza estaba cancelada, y aquí solo cambia la proporción. Pero lo dejo anotado como duda, porque pesa mucho: con el modelo de prueba, sin ella el F1 baja de 0,678 a 0,590, así que si fuera fuga el resultado real sería bastante peor. Con `booking_changes` pasa algo parecido, en menor medida.
- **Quitar los duplicados es una suposición.** Sin un identificador de reserva no sé si algunas de esas filas eran reservas distintas de un mismo grupo.
- **No ajusté los hiperparámetros de la red neuronal,** porque cada entrenamiento es bastante lento. Con más tiempo se podría intentar.
- **El modelo encuentra relaciones, no causas:** que un agente tenga más cancelaciones no quiere decir que el agente sea la causa.

11. Bonus técnicos implementados

Bonus	Dónde está	Resultado
Optimización de hiperparámetros con GridSearchCV	<code>config.BUSQUEDA = "grid"</code> y <code>espacio_busqueda()</code> de cada modelo en <code>src/model_trainer.py</code>	Con la búsqueda, el F1 en validación cruzada mejora en el árbol (0,637 → 0,651), el Random Forest (0,654 → 0,693) y XGBoost (0,686 → 0,697). La regresión logística queda igual (0,542). Las rejillas incluyen los valores por defecto de cada modelo, así que la búsqueda nunca lo deja peor que sin ella. La red no entra en la búsqueda porque es muy lenta.

Anexos

- [Diccionario de variables](#): las 32 columnas del CSV.
- [Guion de la práctica](#): el enunciado.
- Notebooks finales:
 - [notebooks/finales/eda_final.ipynb](#): el análisis exploratorio.
 - [notebooks/finales/comparativa_modelos.ipynb](#): la comparación de modelos y las figuras.
- Notebooks de pruebas: [notebooks/exploracion/](#). En `pruebas_modelos.ipynb` están las pruebas de las fugas, de country, de `booking_changes` y de la red.